

Bare-Metal I2C Sensor Dashboard

Nathaniel Ross

Task A: I2C Init

Do: Configure the I2C1 peripheral registers so the bus is ready to use.

Code:

main.c extras:

```
#include "i2c.h"
i2c1_init();
```

i2c.h

```
#ifndef I2C_H
#define I2C_H
```

```
#include "stm32f4xx.h"
```

```
void i2c1_init(void);
```

```
#endif
```

i2c.c

```
#include "i2c.h"
```

```
void i2c1_init(void) {
    // clock enables
    RCC->AHB1ENR |= (1 << 1);    // GPIOB
    RCC->APB1ENR |= (1 << 21);    // I2C1

    // set PB8 and PB9 to alternate function mode (10)
    GPIOB->MODER &= ~(0b1111 << 16); // clear
    GPIOB->MODER |= (0b1010 << 16);   // set

    // set PB8 and PB9 to open-drain
    GPIOB->OTYPER |= (0b11 << 8);

    // pull-up pull-down mode
    GPIOB->PUPDR &= ~(0b1111 << 16); // clear
    GPIOB->PUPDR |= (0b0000 << 16);   // set (to zero)

    // set alternate function register
    GPIOB->AFR[1] &= ~(0xFF); // clear
    GPIOB->AFR[1] |= (0x44);   // set to AF4

    // sets peripheral clock frequency
    I2C1->CR2 &= ~(0b111111); // clear
    I2C1->CR2 |= (0b101010);   // 42 MHz
}
```

```

// set the SCL clock
I2C1->CCR &= ~(0b111111111111); // clear
I2C1->CCR |= (0b000011010010); // set to 100kHz

// set the rise time
I2C1->TRISE &= ~(0b111111); // clear
I2C1->TRISE |= (0b101011); // set to 43ns

// enable peripheral
I2C1->CR1 |= 0b1;
}

```

Results:

No actual results yet. Compiles correctly.

19:57:06 Build Finished. 0 errors, 0 warnings. (took 1s.491ms)

Learned:

- I2C Protocol has a bus, two wires, SCL (clock) and SDA (data). The master (STM32) controls the clock, and the slave (in this project, the MPU-6500 sensor) responds when addressed
- All devices on the bus hear transmissions, but only addressed device responds
- Uses **open-drain** with pull-up resistors so that multiple devices can be connected
- **Pull-up** resistors are active low, meaning the activation creates the short-circuit to ground and turn on
- **Pull-down** resistors are active high, meaning the activation creates the short-circuit to high voltage and turn on
- STM32 peripherals are clock gated to save power, this is why RCC was used to enable clocks for GPIOB and I2C1
- PB8 and PB9 were configured such that
 - MODER set to 10 (alternate function mode) for I2C peripheral usage
 - OTYPER set to open-drain because of I2C
 - PUPDR set to none because sensor already has these resistors
 - AFR (alternate function) set to AF4 because these are specifically mentioned in the STM32F4xxE datasheet
- I2C peripherals were configured such that
 - CR2 was the peripheral clock and it was told the APB1 clock speed of 42MHz
 - CCR is the SCL clock and it was set to 100kHz
 - TRISE is the maximum rise time for the peripheral, set to 43ns
 - CR1 PE was set to 1 to enable

Task B: WHO_AM_I

Do: Create an I2C read function that reads the MPU-6500's device address and sends it back via UART Shell created in the last project.

Code:

i2c.c:

```
// function to read any register from an I2C device
uint8_t i2c_read(uint8_t device_address, uint8_t regular_address, uint8_t*
result)
{
    // create START condition
    I2C1->CR1 |= (1 << 8);
    uint32_t timeout = 10000;
    // checks SB if start condition created
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to write mode
    I2C1->DR = device_address << 1 | 0;
    timeout = 10000;
    // checks ADDR if address successfully sent and ACKed
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // set register address to read from
    I2C1->DR = regular_address;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // create another START condition
    I2C1->CR1 |= (1 << 8);
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
}
```

```

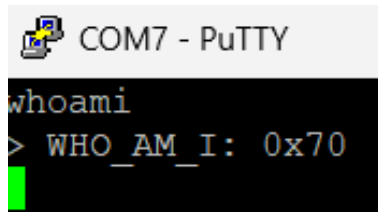
    // set device register to read mode
    I2C1->DR = device_address << 1 | 1;
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // disable ACK because only reading one byte
    I2C1->CR1 &= ~(1 << 10);
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // generate STOP condition
    I2C1->CR1 |= (1 << 9);
    timeout = 10000;
    // check RXNE for data ready to be read
    while(!(I2C1->SR1 & (1 << 6))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // read data register
    *result = I2C1->DR;
    return I2C_OK;
}

main.c:
else if(strcmp((char*)cmd, "whoami") == 0)
{
    uint8_t result = 0;
    char msg[32];
    uint8_t err = i2c_read(0x68, 0x75, &result);
    if(err != I2C_OK)
    {
        snprintf(msg, sizeof(msg), "> Error\r\n");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
    else
    {
        snprintf(msg, sizeof(msg), "> WHO_AM_I: 0x%02X\r\n", result);
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
}

```

Results:



```
COM7 - PuTTY
whoami
> WHO_AM_I: 0x70
```

Learned:

- How to write a complete I2C read function
- Read the WHO_AM_I register (standard in MPU sensors) and got 0x70 back. This confirms the GPIO, peripheral config, and bus protocol all works
- There needs to be a START before writing and reading
- The read/write bits are the LSB of the address byte, 0 for write and 1 for read
- A status flag is polled (wait for potential timeout) in SR1 after each step
- NACK before last byte tells slave (MPU) to not send more data

Task C: Sensor Init

Do: Wake the MPU from sleep, configure accel/gyro range and sample rate

Code:

i2c.c:

```
uint8_t i2c_write(uint8_t device_address, uint8_t regular_address, uint8_t
byte_to_write)
{
    // create START condition
    I2C1->CR1 |= (1 << 8);
    uint32_t timeout = 10000;
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to write mode
    I2C1->DR = device_address << 1 | 0;
    timeout = 10000;
    // checks ADDR if address successfully sent and ACKed
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // set register address to write
    I2C1->DR = regular_address;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set byte to write
    I2C1->DR = byte_to_write;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // generate STOP condition
    I2C1->CR1 |= (1 << 9);
```

```

        return I2C_OK;
    }

mpu.c:
#include "mpu.h"

#define MPU_ADDR 0x68

void mpu_init(void)
{
    // wake up MPU from sleep
    i2c_write(MPU_ADDR, 0x6B, 0x00);
    // configure sensor registers
    i2c_write(MPU_ADDR, 0x1B, 0x00);    // gyro
    i2c_write(MPU_ADDR, 0x1C, 0x00);    // accel
}

mpu.h:
#ifndef MPU_H
#define MPU_H

#include "stm32f4xx.h"
#include "i2c.h"

void mpu_init(void);

#endif

```

Results:

No results, but compiles correctly

22:00:11 Build Finished. 0 errors, 0 warnings. (took 1s.616ms)

Learned:

- Writing does not need multiple STARTs
- Created specific "mpu.c" and "mpu.h" files for the MPU 6500 sensor
- How to wake up the sensor by clearing power management bit
- Setting gyroscope and accelerometer to default state requires clearing bits

Task D: Burst Read

Do: Read all 14 bytes of sensor data in one transaction and convert raw values to physical units (acceleration, temperature, gyro)

Code:

main.c:

```
    else if(strcmp((char*)cmd, "read") == 0)
    {
        int16_t data[7];
        char msg[64];
        uint8_t err = mpu_read(data);
        if(err != I2C_OK)
        {
            snprintf(msg, sizeof(msg), "> Error\r\n");
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
        }
        else
        {
            MPU_Data mpu_data;
            mpu_convert(data, &mpu_data);

            snprintf(msg, sizeof(msg), "> Accel X: %.2f\r\n",
mpu_data.accel_x);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Accel Y: %.2f\r\n",
mpu_data.accel_y);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Accel Z: %.2f\r\n",
mpu_data.accel_z);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Temp (C): %.2f\r\n",
mpu_data.temp_c);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Gyro X: %.2f\r\n",
mpu_data.gyro_x);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Gyro Y: %.2f\r\n",
mpu_data.gyro_y);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Gyro Z: %.2f\r\n",
mpu_data.gyro_z);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
```



```

    }
}

i2c.c:
uint8_t i2c_read_burst(uint8_t device_address, uint8_t regular_address,
uint8_t* buffer, uint8_t length)
{
    // create START condition
    I2C1->CR1 |= (1 << 8);
    uint32_t timeout = 10000;
    // checks SB if start condition created
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to write mode
    I2C1->DR = device_address << 1 | 0;
    timeout = 10000;
    // checks ADDR if address successfully sent and ACKed
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // set register address to read from
    I2C1->DR = regular_address;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // create another START condition
    I2C1->CR1 |= (1 << 8);
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to read mode
    I2C1->DR = device_address << 1 | 1;
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)

```

```

        return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // read each byte
    for(int i = 0; i < length-1; i++)
    {
        timeout = 10000;
        // check RXNE for data ready to be read
        while(!(I2C1->SR1 & (1 << 6))) {
            if(--timeout == 0)
                return I2C_ERROR;
        }
        // read data register
        buffer[i] = I2C1->DR;
    }

    // disable ACK for last byte
    I2C1->CR1 &= ~(1 << 10);
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // generate STOP condition
    I2C1->CR1 |= (1 << 9);
    timeout = 10000;
    // check RXNE for data ready to be read
    while(!(I2C1->SR1 & (1 << 6))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // read last byte
    buffer[length-1] = I2C1->DR;

    // enable ACK
    I2C1->CR1 |= (1 << 10);
    return I2C_OK;
}

mpu.c:
uint8_t mpu_read(int16_t* data)
{
    // reads all 14 bytes for gyro, temp, and accel data
    uint8_t buffer[14];
    if (i2c_read_burst(MPU_ADDR, 0x3B, buffer, 14) == 0)
    {
        for(int i = 0; i < 14; i+=2)

```

```

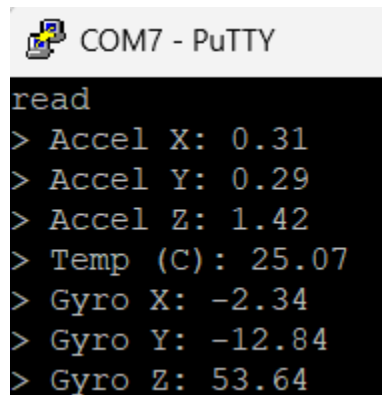
        {
            data[i/2] = buffer[i+1] | buffer[i] << 8;
        }

        return I2C_OK;
    }
    else
        return I2C_ERROR;
}

void mpu_convert(int16_t* raw, MPU_Data* out)
{
    // converts MPU data into physical units
    out->accel_x = raw[0] / 16384.0f;
    out->accel_y = raw[1] / 16384.0f;
    out->accel_z = raw[2] / 16384.0f;
    out->temp_c = (raw[3] / 333.87f) + 21.0f;
    out->gyro_x = raw[4] / 131.0f;
    out->gyro_y = raw[5] / 131.0f;
    out->gyro_z = raw[6] / 131.0f;
}

```

Results:



```

read
> Accel X: 0.31
> Accel Y: 0.29
> Accel Z: 1.42
> Temp (C): 25.07
> Gyro X: -2.34
> Gyro Y: -12.84
> Gyro Z: 53.64

```

Learned:

- Reading many bytes in one transaction requires ACKing each received byte and then NACKing the last received byte
- Assembling the MPU data values required converting the 14 bytes into 7 signed integer (int16_t) values
- How to convert the raw MPU data into actual units by looking at the MPU 6500 datasheet
- Added ACK re-enable to end of i2c read functions so that data can be continuously read
- Burst read (14 bytes at once) is much more efficient than 14 individual reads (don't have to START each time)

Task E: Stream read

Do: Add a “stream” command to get frequent readings of MPU sensor data, and “stop” command to end the stream of data

Code:

main.c:

Inside main(void) while loop:

```
// BLOCKING
// need timers to create non-blocking
if(streaming == 1)
{
    print_mpu_data();
    HAL_Delay(1000);
}
```

Inside process_command(cmd) function:

```
// reads accel, temp, and gyro data from MPU
else if(strcmp((char*)cmd, "read") == 0)
{
    print_mpu_data();
}
// constantly streams MPU sensor data
else if(strcmp((char*)cmd, "stream") == 0)
{
    streaming = 1;
    char msg[64];
    snprintf(msg, sizeof(msg), "> Streaming MPU Data\r\n");
    HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
// stops stream of MPU sensor data
else if(strcmp((char*)cmd, "stop") == 0)
{
    streaming = 0;
    char msg[64];
    snprintf(msg, sizeof(msg), "> Stopping Data Stream\r\n");
    HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
```

Results:

```
stream
> Streaming MPU Data
> Accel X: -0.08
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 24.93
> Gyro X: -3.50
> Gyro Y: 8.60
> Gyro Z: -0.95
> Accel X: -0.09
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.07
> Gyro X: -3.34
> Gyro Y: 8.82
> Gyro Z: -1.05
s> Accel X: -0.08
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 24.98
> Gyro X: -3.05
> Gyro Y: 8.89
> Gyro Z: -1.58
t> Accel X: -0.09
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.12
> Gyro X: -3.53
> Gyro Y: 9.21
> Gyro Z: -1.25
o> Accel X: -0.07
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.27
> Gyro X: -3.31
> Gyro Y: 8.68
> Gyro Z: -1.24
p> Accel X: -0.07
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.46
> Gyro X: -3.46
> Gyro Y: 9.00
> Gyro Z: -1.24
> Stopping Data Stream
```

Learned:

- Adding the stream was possible by implementing a new “streaming” flag. This flag would have MPU data be fetched when set to 1, and nothing would happen when set to 0.
- This added a new issue: The “stream” function floods the console AND is **blocking** (uses polling). This is shown when typing the “stop” command, each letter of that command is processed as each MPU data stream is printed. This takes multiple seconds for that command to process
- To fix this, need to implement **timers** to allow for **non-blocking** streaming of sensor data
- This will be implemented in the next project

What was learned overall:

- Project successfully added to previous UART Shell project by adding new commands “whoami” “read” “stream” and “stop” to read from an I2C connected MPU sensor
- Used bare-metal I2C communication by configuring registers appropriately based on STM32 F4xxE datasheet
- I2C has master (STM32) and slave (MPU-6500) devices, SCL clock line, and SDA data line. The master controls the clock. Address bits, register addresses, data bytes, ACKs, etc travel on the data line. Both are open-drain, meaning many devices can be connected at once
- There is a START before read and write, with read requiring two START conditions
- After each step for reading and writing, there is a status register flag that confirms what has been done, this is polled with a timeout meaning the previous step did not work
- The slave sends ACK after it receives an address, register, or data byte, and the master sends a NACK after the final byte to tell the slave to stop sending data
- The implementation of the I2C protocol allows for real-time readings of data from the MPU-6500
- Currently, streaming the data is blocking, meaning it takes away from processing new commands for large chunks of time. This needs to be changed with timers and interrupts with the next project.
- One of the biggest bugs I faced in this project was that I forgot to re-enable the ACK bit after single-byte reads. This bug made it so multiple commands could not be processed on startup. To fix this, I added ACK re-enable to the end of the `i2c_init` and `read` functions in the `i2c.c` file.
- In all, this project taught me the core concepts of the I2C communication protocol and how it is configured on the register level